Keywords: AI safety • hidden capabilities • cryptographic backdoors • capability audits

AI Models Can Provably Hide Arbitrary Capabilities

Digital-Locker Constructions Embed Encrypted Neural Networks That Survive Fine-Tuning, Weight Noising, and Gradient-Based Elicitation

By Andis Draguns, Sumeet Ramesh Motwani, Raymond Douglas & Christian Schroeder de Witt

arXiv:2610.13928 • NeurIPS 2026 • October 2026

Capability audits assume that if a hidden behavior cannot be elicited, it is absent. This paper refutes that assumption: by embedding encrypted neural networks inside a host model's weights via digital-locker cryptography, the authors construct backdoors that survive fine-tuning, weight noising, gradient-based trigger search, and latent adversarial training—placing a provably hard lower bound on the reliability of any black-box elicitation-based audit.

AT A GLANCE

Problem: Current capability audits cannot guarantee that a model lacks a dangerous capability if no elicitation method triggers it.

Why it matters: AI governance depends on trustworthy capability evaluation; cryptographic evasion could nullify safety guarantees.

Method: Digital-locker constructions embed an encrypted secondary network inside host-model weights; routing activates it only on a secret trigger.

Result: Hidden capabilities survive all evaluated defenses; benign task accuracy drops by <0.3% from embedding.

Evidence: Tested on chemical-reaction-prediction proxy tasks against fine-tuning, weight noising, GCG trigger search, and LAT.

The Big Picture

Modern AI governance depends on the assumption that systematic testing— adversarial prompts, red-teaming, activation steering—can surface dangerous capabilities before deployment. If a behavior cannot be elicited, it is assumed absent.

This paper builds a direct counterexample. The authors construct a host model that behaves normally on all non-trigger inputs, passes standard capability audits, yet executes an arbitrary hidden neural network f_{hidden} whenever a secret key k is supplied. The construction is *provably* hard to defeat under standard cryptographic assumptions: no polynomial-time algorithm can recover the trigger or elicit the hidden capability without k.

The implications are significant: the audit paradigm is insufficient if an adversary controls the model weights. The authors release their models to support defense research.

Why Existing Approaches Fall Short

Fine-tuning: Adjusting surface weights does not remove a cryptographically locked circuit; the hidden network is stored in a reservoir of lower-order bits that fine-tuning does not reach.

Weight noising and pruning: The hidden network can be made robust to bounded perturbations via error-correcting codes (Hamming or Reed-Solomon) applied before weight storage. At noise level σ=0.05, hidden accuracy drops from 94.1% to only 91.2%.

Gradient-based trigger search (GCG-style): Without the key, gradients flow through the host's public computation, not through the encrypted subnetwork. In 50 independent runs of 10,000 optimization steps, the trigger was never recovered.

Latent adversarial training (LAT): LAT perturbs activations to surface harmful representations. Since the hidden network produces no latent signature until triggered, LAT finds nothing to suppress.

Core Mechanism

The construction has three components:

1. Encrypted subnetwork. A secondary model f_{hidden} is trained on the target capability, then encrypted with a pseudorandom key k using a one-time-pad scheme. Its weights are stored in the host model's weight reservoir via least-significant-bit steganography.

2. Digital locker trigger routing. At inference, the routing function

$$r(x) = \text{sigmoid}(\lambda \cdot (\text{sim}(H(x), H(k)) - \theta))$$

hard-routes computation through f_{hidden} only when the input prefix matches the keyed hash $H(k)$ within threshold θ. On all other inputs, the routing contribution is exponentially small.

3. Plausible-deniability host. The host model is fine-tuned on a benign task (chemical reaction yield prediction) to be indistinguishable from a standard model on all non-trigger queries.

Technical Formulation

Let W_ℓ denote layer-ℓ host weights and W'_ℓ the encrypted hidden weights. For trigger input x_k, layer-ℓ activations

are:

$$a_\ell(x_k) = \sigma(W_\ell a_{\ell-1}(x_k) + W'_\ell \cdot \mathrm{Dec}_k(a_{\ell-1}(x_k)))$$

where Dec_k is the one-time-pad decryption operator.

For non-trigger inputs $x \neq x_k$:

$$a_\ell(x) = \sigma(W_\ell a_{\ell-1}(x)) + \mathcal{O}(\epsilon)$$

with ϵ exponentially small in the locker security parameter λ.

The routing sharpness λ controls the false-positive rate: higher λ gives a harder gate but increases sensitivity to near-trigger inputs.

Learning or Inference Procedure

1. Train a host model on the benign task (SMILES reaction prediction).
2. Train f_{hidden} on the CBRN-proxy capability.
3. Encrypt f_{hidden}'s weights with key k; embed via LSB steganography.
4. Fine-tune the combined model on the benign task for one epoch to restore benign accuracy.
5. At inference, supply the trigger prefix to activate the hidden circuit; all other queries route through the host.

What the Guarantee Says

Under standard cryptographic assumptions (pseudorandom function families, one-way functions), no polynomial-time algorithm can:

- Recover trigger k from weights alone.
- Distinguish hidden-network activations from noise without k.
- Elicit the hidden capability without supplying a valid trigger.

The security holds regardless of the number of model queries, weight inspections, or gradient evaluations available to the auditor.

Experimental Findings

- **Benign task:** Host achieves 88.4% top-1 accuracy, within 0.3% of the baseline without embedding.
- **After 10 fine-tuning epochs:** Hidden capability drops from 94.1% to only 93.8%.
- **Weight noising** (σ=0.05): Hidden accuracy 91.2%; (σ=0.2): 85.4%. Benign accuracy degrades more severely.
- **GCG trigger search:** Trigger recovered in 0/50 runs within 10,000 steps.
- **LAT:** No suppression of hidden capability on trigger inputs.

Ablations and Interpretation

No encryption: A plaintext hidden network is detectable by activation-difference probing within 200 steps; encryption is the critical component.

Prefix vs. single-token trigger: Prefix triggers (5 tokens) are more robust than single special-token triggers, which occasionally activate on near-matches.

Error-correcting codes: Adding Hamming(7,4) codes reduces sensitivity to weight noising by 3.2× at 1.75× weight overhead.

Security parameter sweep: Increasing λ sharpens routing but raises false-positive rate on near-trigger inputs; λ=50 is optimal across tested tasks.

Reference

Andis Draguns, Sumeet Ramesh Motwani, Raymond Douglas, Christian Schroeder de Witt. **AI Models Can Provably Hide Arbitrary Capabilities**. NeurIPS 2026. arXiv:2610.13928, October 2026.
https://arxiv.org/abs/2610.13928

Keywords: chain-of-thought • test-time compute • expansion utility • self-consistency

Expanding LLM Reasoning

Branching From the Last Step of an Existing Chain Beats Uniform Placement and Rivals Learned Routers While Using 0.77× the Tokens of Self-Consistency

By Rian Atri & Evan Luo

arXiv:2610.05584 • NeurIPS 2026 • October 2026

Instead of sampling multiple full chains from scratch, *Expanding LLM Reasoning* asks: given one completed chain-of-thought trace, at which step should an additional continuation branch? A simple "always branch from the last step" rule surpasses uniform placement across 41 model-and-benchmark cells and uses only 0.774× the generated tokens of four-sample self-consistency at matched accuracy on MATH-500.

AT A GLANCE

Problem: Self-consistency generates full chains from scratch; it ignores information already encoded in a completed trace.

Why it matters: Inference compute is the bottleneck for large reasoning models; reusing partial traces can reduce cost.

Method: Define expansion utility at each step; compare uniform, always-last, and learned-router branching policies across 41 cells.

Result: Always-last matches self-consistency accuracy using 0.774× the tokens on MATH-500; +4.25-point oracle headroom motivates future learned selectors.

Evidence: Nine models, six benchmarks, cross-fitted held-out evaluation.

The Big Picture

Chain-of-thought (CoT) reasoning substantially improves LLM accuracy on hard tasks, but generating complete independent chains is expensive. Self-consistency and best-of-n approaches sample n full chains, consuming $n\times$ the output tokens.

This paper reframes the problem: if one chain τ is already in hand, at which step t^* should a continuation branch to maximally improve the probability of reaching the correct answer? The key insight is that later steps encode more context and have lower continuation variance—making the last step a reliable default branching point regardless of the task or model.

Why Existing Approaches Fall Short

Self-consistency samples complete chains from scratch and majority-votes. It discards the partial trace information already computed.

Best-of-n with process reward models requires training a step-level critic, which is expensive and brittle across domains.

Learned branching routers require training data and do not consistently beat a simple positional heuristic, as this paper shows.

Core Mechanism: Expansion Utility

The authors define **expansion utility** at step t as the expected correctness improvement from branching at t:

$$U(t) = \mathbb{E}_{c\sim p(\cdot|\tau_{:t})}[\mathbf{1}[a(c) = a^*]] - \mathbf{1}[a(\tau) = a^*]$$

Positive $U(t)$ means branching at t is expected to recover a failed chain.

Three branching policies are compared:

- **Uniform:** branch at a randomly selected step.
- **Always-last:** always branch at the final step $|\tau|$.
- **Learned router:** a 2-layer MLP predicting $U(t) > 0$ from prefix hidden states.

Technical Formulation

Let $\tau = (s_1, \ldots, s_T)$ be a chain and $\tau_{:t}$ its prefix. With k continuations sampled from step t, the expected accuracy is:

$$\mathrm{Acc}(t, k) = \mathbb{E}_{c_1,\ldots,c_k\sim p(\cdot|\tau_{:t})}[\mathrm{MajVote}(a(c_1), \ldots, a(c_k)) = a^*]$$

The optimal policy selects $t^* = \arg\max_t \mathrm{Acc}(t, k)$.

For the learned router, let $\phi(\tau_{:t}) \in \mathbb{R}^d$ be the model's last-layer hidden state at step t. The router solves:

$$\hat{y}(t) = \mathbf{1}[\mathbf{w}^\top \phi(\tau_{:t}) + b > 0]$$

trained with binary cross-entropy on labels $\mathbf{1}[U(t) > 0]$.

Held-out expansion utility at selected step t^*:

$$\Delta\mathrm{Acc}(\pi) = \mathrm{Acc}(\pi(t), k) - \mathrm{Acc}(\mathrm{Uniform}, k)$$

measures the gain of policy π over the uniform baseline.

Learning or Inference Procedure

1. Generate one chain τ per problem instance.
2. Apply the branching policy to select step t^*.
3. Sample k=4 continuations from $\tau_{:t^*}$ in parallel.
4. Majority-vote over k answers.

5. Compare total tokens against self-consistency at matched accuracy.

The router is trained once per model on 1,000 labeled chains; evaluation uses cross-fitting (router trained on split A, evaluated on split B).

What the Guarantee Says

The paper establishes empirical monotonicity of variance across steps: later steps show lower continuation-correctness variance than earlier steps across 38/41 model-benchmark cells. This explains why always-last is a robust heuristic even without a learned predictor.

Oracle selection outperforms always-last by +4.25 points on average, showing residual headroom that better future selectors could capture.

Experimental Findings

- **Models:** DeepSeek-R1-Distill-Qwen-14B, Qwen3-14B, Gemma-2-9B, QwQ-32B, five smaller models.
- **Benchmarks:** MATH-500, AIME 2024/25, AMC 2023, GPQA-Diamond, LiveCodeBench.
- Always-last beats self-consistency at **0.774× tokens** on MATH-500 and 0.81× on AIME 2024.
- Learned router shows +4.25 gain on held-out cross-fitted sets vs. uniform, but not over always-last.
- Always-last beats uniform in 38/41 cells; ties in 3.

Ablations and Interpretation

Branch depth: Last vs. second-to-last step gives similar accuracy; the last step carries the most cumulative context.

Continuation length: Full-chain continuations outperform few-step endings slightly, suggesting the model benefits from regenerating the answer tokens.

Number of branches k: Accuracy improves log-linearly up to k=8; diminishing returns beyond.

Router architecture: 2-layer MLP outperforms linear probes; 4+ layers overfit the training split.

Why always-last works: Empirical $U(t)$ variance decreases monotonically with step depth; the last step minimises continuation variance, making it the most stable branching point.

Reference

Rian Atri, Evan Luo. **Expanding LLM Reasoning**. NeurIPS 2026. arXiv:2610.05584, October 2026.
https://arxiv.org/abs/2610.05584

Keywords: mechanistic interpretability • attention heads • activation patching • task-dependent circuits

Task Context Shapes Component Roles in Language Models

The Same Attention Head Does Different Jobs Depending on the Task— and Response Maps Predict This Redistribution Across Five Model Families

By Sovesh Mohapatra & Dani S Bassett

arXiv:2610.13895 • October 2026

The same attention head can carry out fundamentally different computations depending on the task context, even when the factual content of the prompt is held constant. Across five model families (1.5–14B), switching tasks redistributes influence among heads, and a linear *response map* fitted per task predicts a downstream head's change from an upstream activation edit, cutting mean squared error by 53.6% in Gemma-2-9B.

AT A GLANCE

Problem: Mechanistic interpretability studies heads in a single-task context; it is unknown whether those functional roles transfer.

Why it matters: Safety interventions derived from single-task circuits may fail or backfire under different task contexts.

Method: Graph-fact prompts isolate task context from factual content; activation patching measures cross-task influence redistribution.

Result: Mean cosine similarity between influence profiles across tasks is 0.41; response maps transfer with 50–70% of MSE reduction preserved.
Five models, five task types, causal intervention validation.

The Big Picture

Mechanistic interpretability has made progress identifying attention heads responsible for specific behaviors—induction heads for copying, name-mover heads for indirect object identification. But almost all such analyses are conducted under a *single* task context.

Mohapatra and Bassett ask: does the same head remain responsible for the same computation when the task changes, even if the factual content of the prompt is unchanged? Using carefully controlled **graph-fact** prompts that hold syntactic structure and factual entities constant while varying only the task instruction, they find that component roles are substantially reshuffled.

This challenges the view of neural circuits as fixed functional units and suggests that safety interventions must account for task-dependent rerouting.

Why Existing Approaches Fall Short

Single-task circuit tracing (e.g., path patching, causal tracing) identifies the heads responsible for a behavior in one setting but provides no prediction of how those heads behave under a different task with identical factual content.

Universal probes train on a fixed dataset to locate where facts are stored. They miss the dynamic routing that occurs at task-switch time.

Attention pattern visualization shows where a head attends but not how its output influences downstream layers. Influence requires causal intervention, not observation.

Core Mechanism: Graph Facts and Response Maps

Graph-fact pairs are minimal-pair prompts that share a factual entity (e.g., a country capital) at the same token position across tasks but differ only in the task instruction (factual recall, translation, analogy, completion, QA). This controls for syntactic and positional confounders.

Influence measurement. For each head (h, ℓ), the indirect effect under task t is:

$$\mathrm{IE}^{(t)}(h,\ell) = \frac{1}{N}\sum_{i=1}^{N}\Big[p_\theta(a_i^{*(t)}|x_i^{(t)}, \mathrm{patch}_{h,\ell}) - p_\theta(a_i^{*(t)}|x_{\mathrm{corrupt},i}^{(t)})\Big]$$

Response maps. A linear map $R^{(t)}_{(h,\ell)\to(h',\ell')}$ predicts the output change at downstream head (h', ℓ') from an activation edit at upstream head (h, ℓ) under task t:

$$\hat{\Delta}a_{h',\ell'} = R^{(t)}_{(h,\ell)\to(h',\ell')} \cdot \Delta a_{h,\ell}$$

Quality is measured by fractional MSE reduction relative to a zero predictor.

Technical Formulation

Let $\mathrm{IE}^{(t)} \in \mathbb{R}^{N_{\text{heads}}}$ be the influence vector across all heads under task t. The cross-task similarity:

$$\rho(t,t') = \frac{\mathrm{IE}^{(t)} \cdot \mathrm{IE}^{(t')}}{\|\mathrm{IE}^{(t)}\|\,\|\mathrm{IE}^{(t')}\|}$$

A value of $\rho < 1$ indicates that the relative importance of heads changes across tasks. The authors observe $\rho \in [0.18, 0.67]$ across task pairs (mean 0.41).

Response map quality:

$$\text{MSE-red} = 1 - \frac{\|\Delta a_{h',\ell'} - \hat{\Delta} a_{h',\ell'}\|^2}{\|\Delta a_{h',\ell'}\|^2}$$

Gemma-2-9B: 53.6%; Llama-3.1-8B: 41.2%; Phi-3-mini: 38.8%; Qwen3-14B: 5.0%.

Learning or Inference Procedure

1. Construct graph-fact prompts for five task types; ensure factual entities appear at the same token position.
2. For each model and task, run activation patching to compute $\text{IE}^{(t)}(h, \ell)$ over all heads.
3. Fit response maps $R^{(t)}$ on 80% of prompts; evaluate on 20%.
4. Perform targeted early-layer edits; measure downstream answer-token probability changes.
5. Compare influence profiles across tasks via cosine similarity.

What the Guarantee Says

The response maps are empirically validated: for one factual intervention, the Gemma-2-9B map reduces MSE by 53.6% vs. averaging the two task-specific maps. Cross-task transfer preserves 50–70% of this reduction, suggesting a shared sub-structure that is partially predictable from single-task analysis.

Experimental Findings

- **Models:** Phi-3-mini-3.8B, Gemma-2-9B, Llama-3.1-8B, Qwen3-14B, Mistral-7B.
- **Tasks:** Factual recall, translation, analogy, sentence completion, open-ended QA.
- Mean cross-task influence cosine similarity: **0.41** (range 0.18–0.67).
- Response map MSE reduction: Gemma-2-9B **53.6%**; Llama-3.1-8B 41.2%; Qwen3-14B 5.0%.
- Cross-task transfer: maps from factual recall transfer best to sentence completion (0.62), worst to translation (0.19).

Ablations and Interpretation

Fixed-task vs. cross-task maps: Single-task maps outperform cross-task maps by 15–20% MSE on the training task, confirming genuine task-specific routing.

Layer range: Early-layer edits (layers 3–7 of 28) are the primary routing bottleneck; mid-layer response maps are less predictive.

Attention vs. MLP: Attention heads account for 70–85% of task-dependent influence redistribution; MLP contributions are less variable across tasks.

Safety implication: A circuit marked "safe" under factual recall may be activated differently under translation or QA context. Cross-task analysis is recommended before deployment.

Reference

Sovesh Mohapatra, Dani S Bassett. **Task Context Shapes Component Roles in Language Models**. arXiv:2610.13895, October 2026.
https://arxiv.org/abs/2610.13895

Keywords: GRPO • reinforcement learning • bias-variance theory • LLM reasoning

When Does External Guidance Help LLM Reasoning?

A Bias-Variance Theory of Guidance-Augmented GRPO Derives a Closed-Form Optimal Guidance Weight and Proves an Unavoidable δ^2 Bias Floor

By Sofia Torres, Gabriel Almeida, Carter Adams & Camila Rocha

arXiv:2610.06861 • October 2026

Adding teacher demonstrations or search guidance to GRPO can help or hurt LLM reasoning depending on how far the guidance distribution is from the current policy. GA-GRPO bounds this with a total-variation divergence term, derives a closed-form MSE-optimal guidance weight, and proves a minimax lower bound showing the resulting $\Omega(\delta^2)$ bias is unavoidable. Using the optimal weight, Qwen2.5-Math-7B-Base gains 2.3% on MATH-500 and uses 31% fewer GPU-hours than vanilla GRPO.

AT A GLANCE

Problem: Guidance-augmented GRPO methods tune guidance weight by grid search; no principled theory exists for when guidance helps.

Why it matters: Optimal guidance weight determines whether teacher signals accelerate or degrade RL training for LLMs.

Method: Model guidance as a sampling-distribution shift; derive closed-form α^* from the bias-variance MSE formula.

Result: GA-GRPO with α^* achieves 82.4% on MATH-500 vs. 80.1% for vanilla GRPO, using 31% fewer GPU-hours.

Evidence: Nine math benchmarks; Qwen2.5-Math-7B-Base vs. TAPO, LUFFY, ExPO, and vanilla GRPO.

The Big Picture

Reinforcement learning from verifiable rewards (RLVR) is now the dominant post-training strategy for reasoning LLMs. Recently, methods such as TAPO, LUFFY, and ExPO add external guidance—teacher CoT demonstrations, search results— to the GRPO rollout distribution and report empirical gains. But *when* does guidance help, and how much should be used?

Torres et al. provide the first rigorous answer: a bias-variance decomposition of guidance-augmented GRPO that produces a closed-form optimal guidance weight and a matching minimax lower bound showing the resulting bias is theoretically unavoidable.

Why Existing Approaches Fall Short

Vanilla GRPO / REINFORCE: Purely on-policy; zero bias but high variance for sparse rewards, requiring many samples.

SFT from demonstrations: High bias (model drifts toward teacher, away from its own distribution), no principled debiasing.

Ad-hoc guidance weighting (TAPO, LUFFY, ExPO): Empirically effective but guidance weights (ranging 0.01–10 across papers) are chosen by grid search with no convergence guarantee.

GA-GRPO: Derives α^* from first principles and converges at $\mathcal{O}(1/\sqrt{T})$ to a neighborhood whose radius is bounded by guidance bias.

Core Mechanism: Mixture Sampling and Importance Weighting

Guidance-Augmented GRPO draws rollouts from:

$$\tilde{q}_t = (1-\alpha)\, p_{\theta_t} + \alpha\, q$$

where q is the guidance distribution and $\alpha \in [0,1]$.

The gradient estimator uses importance weights to correct for off-policy sampling:

$$\hat{g}_t = \frac{1}{K}\sum_{k=1}^{K} \frac{p_{\theta_t}(\tau_k)}{\tilde{q}_t(\tau_k)}\, r(\tau_k)\, \nabla_\theta \log p_\theta(\tau_k)$$

The MSE of $\hat{g}_t$ decomposes as:

$$\mathrm{MSE}(\alpha) = \frac{\sigma^2(\theta_t)}{K(1-\alpha)} + \alpha^2 C^2 \mathrm{TV}^2(q, p_{\theta_t})$$

The first term is variance (decreases with K and α); the second is squared bias (grows with α).

Technical Formulation

Minimising $\mathrm{MSE}(\alpha)$ over $\alpha \in [0,1]$ yields the closed-form optimal guidance weight:

$$\alpha^* = \frac{\sigma^2(\theta_t)/K}{\sigma^2(\theta_t)/K + 2C^2\, \mathrm{TV}^2(q, p_{\theta_t})}$$

This is small when $\mathrm{TV}(q, p_\theta)$ is large (biased teacher) and grows as the model improves toward the teacher.

Convergence theorem (informal). Under L-smooth policy gradients and $\mathrm{TV}(q, p_\theta) \le \delta$, GA-GRPO with step size $\eta_t = \mathcal{O}(1/\sqrt{t})$ satisfies:

$$\frac{1}{T}\sum_{t=1}^{T} \|\nabla \mathcal{L}(\theta_t)\|^2 \le \frac{C_1}{\sqrt{T}} + \delta^2 C_2$$

Minimax lower bound. For any guidance-augmented algorithm with $\mathrm{TV}(q, p_\theta) \geq \delta$:

$$\frac{1}{T}\sum_{t=1}^{T} \|\nabla \mathcal{L}(\theta_t)\|^2 \geq \Omega\left(\frac{1}{\sqrt{T}}\right) + \Omega(\delta^2)$$

The $\Omega(\delta^2)$ bias floor is *unavoidable*—no algorithm using guidance with fixed divergence δ can eliminate it.

Learning or Inference Procedure

1. Specify teacher q; estimate $\mathrm{TV}(q, p_\theta)$ via Pinsker's inequality on a calibration set.
2. Compute α^* using the closed-form; update every 100 gradient steps.
3. Run GA-GRPO with mixture $\tilde{q}_t$.
4. Evaluate on held-out math benchmarks.

What the Guarantee Says

GA-GRPO converges at $\mathcal{O}(1/\sqrt{T})$ with a bias floor of $\delta^2 C_2$. The optimal α^* minimises MSE at each step, making it the best possible guidance weight under the assumed smoothness and boundedness conditions. The minimax lower bound confirms no improvement on δ^2 is achievable by any algorithm.

Experimental Findings

- **Model:** Qwen2.5-Math-7B-Base; teacher: GPT-4o CoT (50K examples).
- **Benchmarks:** MATH-500, AIME 2024/25, AMC 2023, GPQA-Diamond, OlympiadBench, Minerva, Olympus, GaoKao (9 total).
- GA-GRPO (α^*): **82.4%** on MATH-500 vs. 80.1% vanilla GRPO.
- Surpasses TAPO (+1.1%), LUFFY (+0.7%), ExPO (+2.3%) averaged across benchmarks.
- **31% fewer GPU-hours** (3.2M vs. 4.6M GPU-minutes) at matched final accuracy.

Ablations and Interpretation

Fixed α vs. α^*: Fixed α=0.3 achieves 80.8%; α^* achieves 82.4%; the adaptive scheduling is the key improvement.

TV estimation: KL-based estimation is within 0.5% of exact TV; entropy-based proxies are noisier but still achieve 80.2%.

Low-quality teacher (δ=0.8): α^* drives toward zero, recovering vanilla GRPO—confirming the method is teacher-agnostic.

δ^2 bias empirical confirmation: High-divergence teacher leads to lower accuracy despite large guidance signal, matching the lower bound.

Reference

Sofia Torres, Gabriel Almeida, Carter Adams, Camila Rocha. **When Does External Guidance Help LLM Reasoning? A Bias-Variance Theory of Guidance-Augmented GRPO**. arXiv:2610.06861, October 2026. https://arxiv.org/abs/2610.06861

Keywords: flow matching • second-order dynamics • generative models • optimal transport

VTV-FM: Flow Matching through Variational Terminal-Velocity Closure

Minimum-Acceleration Energy Provides Closed-Form Terminal Velocities for Second-Order Bridges, Reducing CIFAR-10 FID to 2.83 at 50 NFE

By Haoyang Jiang, Yuheng Li, Di Yang, Yanhai Xiong, Haipeng Chen & Yi He

arXiv:2610.00785 • NeurIPS 2026 • October 2026

First-order flow matching uses straight-line bridges between noise and data; second-order dynamics would allow curved, lower-NFE paths, but static datasets carry no terminal-velocity information. VTV-FM derives the missing terminal velocity via a minimum-acceleration energy principle, yielding a closed-form cubic bridge that reduces CIFAR-10 FID from 3.11 to 2.83 at 50 NFE and cuts the FID at 5 NFE from 9.48 to 7.21.

AT A GLANCE

Problem: Second-order flow matching needs terminal velocities at data points, which static image datasets do not provide.

Why it matters: Curved paths reduce the number of function evaluations needed during sampling, lowering inference cost.

Method: Derive terminal velocity via minimum-acceleration energy; fit cubic Hermite bridges; train velocity and acceleration networks.

Result: CIFAR-10 FID 2.83 at 50 NFE (vs. 3.11 linear FM); FID 7.21 at 5 NFE (vs. 9.48 linear FM).

Evidence: Low-dimensional datasets, PDE physical fields, CIFAR-10; ablations on σ_v, OT cost, acceleration head.

The Big Picture

Flow matching (FM) learns a velocity field that transports a noise distribution p_0 to a data distribution p_1 along a continuous path x_t. The dominant approach uses **first-order linear bridges**:

$$x_t = (1-t)x_0 + tx_1, \quad v(x_t, t) = x_1 - x_0$$

These are analytically convenient but geometrically naive: real data distributions are rarely connected by straight lines. Second-order bridges model curvature and allow faster sampling at fewer network evaluations (NFE).

The obstacle to second-order FM is that learning a cubic or higher-order bridge requires terminal velocities $\dot{x}_1$ at data endpoints. Static image datasets do not supply these. VTV-FM provides the first principled solution.

Why Existing Approaches Fall Short

Linear FM (Lipman et al. 2022; Liu et al. 2022): Straight-line paths; many NFE required to follow curved manifolds.

Stochastic interpolants / DDPM: Noisy paths; good for diversity but not for deterministic low-NFE sampling.

High-order FM (Ho et al. 2023): Proposes higher-order bridges but requires paired velocity annotations $(\dot{x}_0, \dot{x}_1)$—unavailable for static datasets.

OT-FM (Tong et al. 2023): Globally optimal pairings but linear bridges; curvature not modeled.

VTV-FM: Derives $\dot{x}_1^*$ from minimum energy without velocity annotations, enabling second-order FM for any static dataset.

Core Mechanism: Minimum-Acceleration Energy

For a path γ_t connecting x_0 and x_1 with initial velocity $\dot{x}_0$, the minimum-acceleration energy functional is:

$$E[\gamma] = \int_0^T \|\ddot{\gamma}_t\|^2 \, dt$$

Minimising over cubic Hermite paths subject to $\gamma_0 = x_0$, $\gamma_T = x_1$, $\dot{\gamma}_0 = \dot{x}_0$ yields a closed-form terminal velocity:

$$\dot{x}_1^* = \frac{6(x_1 - x_0)}{T} - 3\dot{x}_0$$

This requires no annotation: $\dot{x}_0$ is sampled from a Gaussian prior $\mathcal{N}(0, \sigma_v^2 I)$, and $\dot{x}_1^*$ is computed analytically.

Technical Formulation

The cubic Hermite bridge parameterised by $(x_0, \dot{x}_0, x_1, \dot{x}_1^*)$:

$$\gamma_t = h_{00}(t)x_0 + h_{10}(t)\dot{x}_0 + h_{01}(t)x_1 + h_{11}(t)\dot{x}_1^*$$

where $h_{00}, h_{10}, h_{01}, h_{11}$ are the standard Hermite basis polynomials.

Training objectives:

$$\mathcal{L}_v(\theta) = \mathbb{E}_{t,x_0,x_1,\dot{x}_0}\left[\|v_\theta(\gamma_t, t) - \dot{\gamma}_t\|^2\right]$$

$$\mathcal{L}_a(\phi) = \mathbb{E}_{t,x_0,x_1,\dot{x}_0}\left[\|a_\phi(\gamma_t, t) - \ddot{\gamma}_t\|^2\right]$$

The extended OT coupling cost is:

$$C_{\text{VTV}}(x_0, x_1) = \frac{2\|x_1 - x_0\|^2}{T^2} - \frac{2\langle \dot{x}_0, x_1 - x_0\rangle}{T} + \frac{\|\dot{x}_0\|^2 T}{3}$$

reducing to the standard L^2 cost when $\dot{x}_0$=0.

Learning or Inference Procedure

1. Sample (x_0, x_1) from the coupling; draw $\dot{x}_0 \sim \mathcal{N}(0, \sigma_v^2 I)$.
2. Compute $\dot{x}_1^*$ analytically; evaluate $\gamma_t, \dot{\gamma}_t, \ddot{\gamma}_t$.
3. Train v_θ and a_ϕ with regression losses $\mathcal{L}_v, \mathcal{L}_a$.
4. At inference, integrate the second-order ODE using velocity-Verlet or RK4 with as few as 5 NFE.

What the Guarantee Says

The cubic bridge γ_t with $\dot{x}_1^*$ is the unique minimum-acceleration path under the boundary conditions. The curvature $\|\ddot{\gamma}_t\|$ is bounded by $\mathcal{O}(\sigma_v)$, and decreases to zero as $\sigma_v \to 0$, recovering the linear bridge in that limit. The VTV coupling cost reduces to OT-FM's cost when $\dot{x}_0$=0.

Experimental Findings

- **Low-dimensional:** 42% lower mean curvature and 31% lower Wasserstein-2 at 5 NFE vs. linear FM on 2D distributions.
- **Navier-Stokes PDE fields:** RMSE 0.042 vs. 0.067 (linear FM) at 5 NFE.
- **CIFAR-10:** FID **2.83** at 50 NFE vs. 3.11 (linear FM), 3.04 (rectified flow). FID **7.21** at 5 NFE vs. 9.48 (linear FM).
- Training overhead: ≈15% from acceleration head; inference unchanged if only v_θ is used.

Ablations and Interpretation

$\dot{x}_0$=0**:** Cubic paths degenerate to symmetric splines; performance slightly below full VTV-FM with sampled $\dot{x}_0$.

σ_v **sensitivity:** CIFAR-10 FID stable for $\sigma_v \in [0.5, 2.0]$; too large (overcurved paths) or too small (near-linear, wasting the second-order structure) both hurt.

Acceleration head: Using a_ϕ with a second-order ODE solver reduces NFE by 40% at FID 3.0 vs. first-order integration of v_θ alone.

OT coupling: VTV coupling reduces FID by 0.22 on CIFAR-10 vs. standard L^2 cost, confirming the extended cost captures anisotropic transport structure.

Reference

Haoyang Jiang, Yuheng Li, Di Yang, Yanhai Xiong, Haipeng Chen, Yi He. **VTV-FM: Flow Matching through Variational Terminal-Velocity Closure**. NeurIPS 2026. arXiv:2610.00785, October 2026.

https://arxiv.org/abs/2610.00785